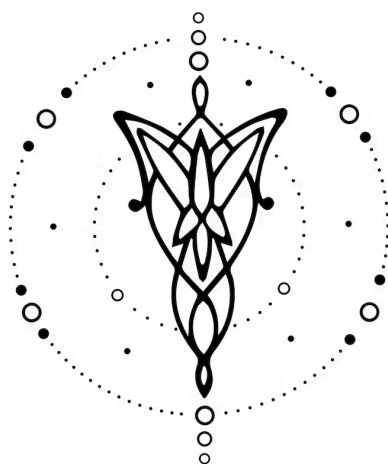




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

V3.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Alexis	Herrera Vegas	64045	aherreravegas@itba.edu.ar
Jeronimo	Esquivel	64756	tesquivel@itba.edu.ar
Andres	Cortese	64612	acortese@itba.edu.ar
Sebastian	Caules	64331	scaules@itba.edu.ar

2. Repositorio

<https://github.com/Alexishvegas/Flex-Bison-Compiler>

3. Dominio

El objetivo del proyecto es desarrollar un lenguaje de programación específico (DSL) que permita definir, visualizar y manipular agendas de eventos en un calendario, ofreciendo una sintaxis expresiva centrada en el dominio del tiempo.

El lenguaje debe posibilitar la creación de calendarios mensuales o anuales, con soporte para eventos puntuales o recurrentes, y ofrecer herramientas para reemplazar, sobrescribir o extender eventos existentes mediante construcciones de alto nivel.

Cada evento define un instante o período temporal (por ejemplo, una clase semanal o una reunión de media hora), junto con atributos descriptivos como nombre, horario, color o enlace.

El resultado de la interpretación del lenguaje será un calendario HTML visualmente representativo, donde los eventos se renderizan con sus propiedades y colores.

El lenguaje incluye:

- **Declaración de zona horaria**, permitiendo especificar contextos temporales (por ejemplo, GMT-3, EST+5 o CET+10, entre otros).
- **Bloques anidados** para agrupar períodos:

```
timezone GMT-3
define color blue #0000FF
year 2025 {
  month August {
    event Taller on Monday and Friday from 08:00 to 10:00 { color blue }
  }
}
```

- **Eventos expresivos** con palabras clave semánticas: event, on, from, to, every, override, till, since.
- **Sobrescritura de eventos** mediante override para redefinir total o parcialmente eventos previos, conservando su horario y contexto.

El propósito final es crear un lenguaje simple, legible y contextualizado, que acerque la definición de cronogramas a un estilo natural.

4. Construcciones

El lenguaje debe ofrecer las siguientes construcciones y reglas:

I. Definición temporal global

- A. `timezone <zona>`: establece el huso horario, p. ej. `timezone GMT-3`.
- B. `year <número> { ... }`: bloque que agrupa todos los eventos de un año.
- C. `month <nombre|número> { ... }`: bloque anidado dentro de `year`, que define los eventos de un mes específico.

II. Eventos

- A. Definición básica:

`event Título on Monday from 10:00 to 11:00 { color red }`

Permite definir un evento con nombre, día y horario.

III. Sobrescritura y reemplazo

`override "Evento"`: redefine un evento previamente declarado, conservando su horario base:

```
override Taller {
  description "Taller especial"
  color lime
}
```

Reemplaza la instancia del evento en una fecha específica o bajo ciertas condiciones.

IV. Colores y estilos

- A. `define <nombre> <valor>` permite registrar colores personalizados

(define cyan #00FFFF).

- V. **Comentarios:** No incluimos comentarios, no creemos que tengan un uso real en nuestro lenguaje.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). **Evento puntual:**
Programa que declara un evento único en una fecha y horario específicos, verificando que se represente correctamente en el calendario.
- (II). **Evento recurrente semanal:**
Programa que declara un evento que ocurre todos los lunes del mes, comprobando que se repita automáticamente en cada ocurrencia correspondiente.
- (III). **Evento con descripción y link:**
Programa que define un evento con una descripción y una URL.
- (IV). **Concatenación de días:**
Programa que declara un mismo evento en múltiples días de la semana (por ejemplo, lunes y viernes), verificando que se genere una única definición y se duplique visualmente en el calendario.
- (V). **Uso de color declarado:**
Programa que utiliza un color personalizado definido previamente mediante una declaración, comprobando que se aplique correctamente al evento.
- (VI). **Múltiples bloques de meses distintos:**
Programa que define eventos en diferentes meses de un mismo año, comprobando que se genere un calendario completo.
- (VII). **Múltiples bloques de meses distintos en diferentes formatos:**
Programa que declara bloques de meses usando tanto el formato en texto (November) como el formato numérico (12 para diciembre).
- (VIII). **Sobrescritura de evento:**
Programa que reemplaza un evento puntual o recurrente, conservando su horario y título, pero modificando sus propiedades particulares como color o descripción.
- (IX). **Programa sin definición de huso horario:**
Programa que no define un huso horario específico y se genera correctamente.
- (X). **Combinación de construcciones:**
Programa que combina múltiples funcionalidades del lenguaje (recurrentes, sobrescritura, propiedades) en un mismo calendario, comprobando que todas las interacciones sean interpretadas correctamente y generen la salida esperada.

Además, los siguientes casos de prueba de **rechazo**:

- (I). **Estructura sintáctica incorrecta:**
Programa que presenta errores en la estructura del código, como llaves o palabras clave ausentes.
- (II). **Color no declarado o inválido:**
Programa que utiliza un color que no fue declarado previamente o con un formato hexadecimal incorrecto.
- (III). **Fecha inexistente:**
Programa que intenta declarar un evento en una fecha no válida, como el 30 de febrero.
- (IV). **Horario incoherente:**
Programa que indica una hora de finalización anterior a la hora de inicio del evento.

- (V). **Número de mes incorrecto:**
Programa que declara un bloque de mes con un número de mes incorrecto, como el mes 13.
- (VI). **Uso de palabra reservada:**
Programa que intenta declarar un color, evento o variable utilizando una palabra reservada del lenguaje.
- (VII). **Sobrescritura inexistente:**
Programa que intenta sobrescribir un evento que no fue declarado previamente.
- (VIII). **Superposición de eventos:**
Programa que intenta declarar eventos que se superponen en el tiempo. No permitimos este comportamiento.

6. Ejemplos

Ejemplo 1

Se define el calendario correspondiente al año **2025** para el mes de **octubre**.

Primero se declaran colores personalizados: blue, magenta, cobalt y lime.

A continuación, se definen los siguientes eventos:

- **Taller:** evento recurrente que ocurre los **lunes y viernes**, de **08:00 a 10:00**, con color **azul**.
- **Seminario Redes:** evento recurrente que ocurre los **miércoles** del mes, de **14:00 a 16:00**, con color **magenta**.
- **Consultas:** evento recurrente que ocurre los **martes y jueves**, de **17:00 a 18:00**, con color **cobalt** (previamente declarado).
- **Taller especial:** sobrescribe (override) el evento “Taller”, agregando la descripción “Taller especial” y cambiando su color a **lime** (también declarado).

```
timezone gmt-3
define color blue    #0000ff
define color cobalt  #0047ab
define color lime    #32cd32
define color magenta #ff00ff
year 2025 {
    month october {
        event taller on Monday and Friday from 08:00 to 10:00 { color blue }
        event seminarioredes on Wednesday from 14:00 to 16:00 { color magenta }
        event consultas on Tuesday and Thursday from 17:00 to 18:00 { color cobalt }

        override taller {
            description "taller especial"
            color lime
        }
    }
}
```

```
}  
}
```

Ejemplo de output en fig. 1

Ejemplo 2

Se crea un calendario correspondiente al año **2025** para el mes de **agosto**.

Dentro de este calendario, se declara un **evento único** llamado “Examen Parcial” el día **10 de agosto**, desde las **09:00 hasta las 11:00 horas**, con color **rojo**.

Posteriormente, se realiza una **sobrescritura** (override) sobre esa misma fecha para modificar la descripción a “Examen Parcial Reprogramado” y el color a **cyan**, manteniendo el horario original.

Este ejemplo permite validar el correcto funcionamiento del mecanismo de sobrescritura, comprobando que se reemplacen únicamente los atributos indicados (descripción y color) sin alterar las demás propiedades del evento original.

```
timezone gmt-3  
define color red   #ff0000  
define color cyan  #00ffff  
  
year 2025 {  
    month august {  
        event examenParcial on 10 from 09:00 to 11:00 { color red }  
  
        override examenParcial {  
            description "examen parcial reprogramado"  
            color cyan  
        }  
    }  
}
```

Ejemplo 3

Se define un bloque **anual 2026** que contiene dos **bloques mensuales**: **marzo** y **abril**.

- En **marzo**, se declara un evento recurrente “**Consultas**” que ocurre **todos los martes** de **17:00 a 18:00**, utilizando un **color declarado** previamente (cobalt).
- En **abril**, se programa “**Reunión General**” como **evento puntual** del día **3** de abril, con un color corporativo declarado.

Este ejemplo valida:

1. el **anidamiento temporal** year { month { ... } },
2. la **recurrencia semanal** en un mes específico,
3. la **declaración y reutilización** de colores personalizados, y
4. la representación de un **evento puntual**

```

timezone gmt-3
define color cobalt    #0047ab
define color corporate #0a84ff

year 2026 {
  month march {
    event consultas on tuesday from 17:00 to 18:00 { color cobalt }
  }

  month april {
    event reuniongeneral on 3 from 09:00 to 12:00 { color corporate }
  }
}

```

2025 CALENDAR YEAR		OCTOBER					MONDAY FIRST DAY OF WEEK	
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday		
29	30	01 Seminario Redes 14:00-16:00	02	03 Taller 08:00-10:00	04	05		
06 Taller 08:00-10:00	07	08	09 Consultas 17:00-18:00	10	11	12		
13	14 Consultas 17:00-18:00	15	16	17 Taller especial 08:00-10:00	18	19		
20 Taller 08:00-10:00	21	22	23 Consultas 17:00-18:00	24	25	26		
27	28 Consultas 17:00-18:00	29	30	31 Taller 08:00-10:00	01	02		

FIG. 1: Output del ejemplo 1 en formato html hecho con un programa de prueba en python

7. Correcciones

Decisiones posteriores al feedback del Stage II:

Se corrigieron todos los errores o detalles marcados en la devolución del Stage II como Warnings, uso del pipeline CI, cantidad de tests, defaults en switch y backtracking. La única observación que no pudimos corregir es la independencia entre Flex y Bison para el ciclo de vida de ciertas variables, duplicadas con strdup. Se intentaron efectuar correcciones pero resultaron en problemas de leaking difíciles de solucionar prolijamente, por lo que decidimos no desperdiciar el tiempo en eso.